



# EE-331

## Project Report

### System Design of Ball and Beam

| <i>Member No.</i> | <i>Name</i>                 | <i>ID</i> |
|-------------------|-----------------------------|-----------|
| #1                | Ahmed Al-Harthy             | 2237094   |
| #2                | Naif Al-Suhaim              | 2036399   |
| #3                | Abdullah Mustafa Alaidaroos | 1937923   |
| #4                | Masoud Saad Albogami        | 2135429   |
| #5                | Abdullah Fallatah           | 2036657   |

# Table Of Content

|       |                                                                 |    |
|-------|-----------------------------------------------------------------|----|
| 1.    | Introduction .....                                              | 3  |
| 2.    | System Modeling .....                                           | 3  |
| 2.1   | Description of the System.....                                  | 3  |
| 2.2   | Relation Between Beam Angle and Motor Angle .....               | 4  |
| 2.3   | Building the Model .....                                        | 4  |
| 2.4   | Transfer Function.....                                          | 5  |
| 3.    | System Analysis .....                                           | 6  |
| 3.1   | Open-Loop Plant Model .....                                     | 6  |
| 3.2   | Poles, Zeros, and Stability .....                               | 6  |
| 3.3   | Time-Domain Behavior to Angle Step Input.....                   | 7  |
| 3.4   | System Type and Steady-State Error (Position Control Loop)..... | 9  |
| 4.    | PID Controller Design .....                                     | 10 |
| 4.1   | Design Objective and Control Structure.....                     | 10 |
| 4.2   | Proportional Control Evaluation.....                            | 10 |
| 4.3   | Proportional–Derivative Controller Design.....                  | 11 |
| 4.3.1 | Adding Derivative Action .....                                  | 11 |
| 4.3.2 | Tuning Procedure .....                                          | 12 |
| 4.4   | Justification for Zero Integral Gain .....                      | 13 |
| 4.5   | Final Controller.....                                           | 13 |
| 5.    | Root Locus Controller Design .....                              | 14 |
| 5.1   | Open-loop root locus .....                                      | 14 |
| 5.2   | Lead-compensator design .....                                   | 15 |
| 6.    | Simulink .....                                                  | 17 |
| 6.1   | ball-and-beam model .....                                       | 17 |

|                                           |    |
|-------------------------------------------|----|
| 6.2 ball-and-beam controller.....         | 17 |
| 6.3 ball and beam simulated results ..... | 18 |
| 7. Conclusion .....                       | 19 |
| 8. References .....                       | 20 |
| Appendix A .....                          | 21 |
| Appendix B .....                          | 22 |
| System analysis code .....                | 22 |
| PID code.....                             | 23 |
| Root Locus Controller Design Code.....    | 24 |

# 1. Introduction

The objective of this project is to analyze and design a control system for a "Ball and Beam" mechanism. The Ball and Beam is a classic unstable system widely used in control engineering to demonstrate feedback control principles. The goal is to stabilize the ball at a specific position along the beam by automatically adjusting the beam's angle. In this report, we present the mathematical modeling of the system, analyze its behavior, and design a controller to stabilize it using MATLAB and Simulink simulations.

## 2. System Modeling

### 2.1 Description of the System

The Ball and Beam system is a standard device used to study control systems. It consists of a long beam that can tilt up and down, with a ball placed on top of it [1].

The mechanism works in the following way:

- A motor is connected to the beam using a lever arm.
- When the motor rotates, it pushes the lever, which causes the beam to tilt.
- When the beam tilts, gravity pulls the ball, causing it to roll along the beam.

The goal of the project is to control the motor so that the beam tilts just enough to keep the ball balanced at a specific position. Because the ball is free to roll, the system is naturally unstable; if the motor stops adjusting the angle, the ball will roll off the end of the beam.

To model the ball-and-beam mechanism, we relate the motor's rotation ( $\theta$ ) to the ball's position ( $x$ ) on the beam. The beam tilts when the motor rotates, and this tilt angle controls the acceleration of the ball.

To ensure our simulation is accurate, we used real physical data taken from a research paper [2]. We built our model using the specific measurements found in that study. This ensures that our simulation behaves exactly like the real-world system described in the research.

TABLE 1: PARAMETERS FOR THE BALL AND BEAM SYSTEM THAT WILL BE USED

| <i>Parameter</i>           | <i>Symbol</i>        | <i>Value</i>                                     | <i>Unit</i>             |
|----------------------------|----------------------|--------------------------------------------------|-------------------------|
| <i>Beam Length</i>         | <b>L</b>             | <b>0.34</b>                                      | <b>m</b>                |
| <i>Lever Arm Offset</i>    | <b>d</b>             | <b>0.0254</b>                                    | <b>m</b>                |
| <i>Ball Mass</i>           | <b>m</b>             | <b>0.064</b>                                     | <b>kg</b>               |
| <i>Ball Radius</i>         | <b>R</b>             | <b>0.0127</b>                                    | <b>m</b>                |
| <i>Ball Inertia</i>        | <b>J<sub>b</sub></b> | <b>3.87*10<sup>-7</sup><br/>kgm<sup>2</sup>)</b> | <b>kg·m<sup>2</sup></b> |
| <i>Gravitational Accel</i> | <b>g</b>             | <b>9.81</b>                                      | <b>m/s<sup>2</sup></b>  |

## 2.2 Relation Between Beam Angle and Motor Angle

When the beam is tilted by a small angle  $\alpha$ , the component of gravity along the beam accelerates the ball. The equation of motion for rolling without slipping is:

$$\alpha = \frac{d}{L} \theta$$

## 2.3 Building the Model

Substituting  $\alpha$  into the ball dynamic equation and taking the Laplace transform:

$$\frac{X(s)}{\theta(s)} = \frac{mgd}{L\left(m + \frac{Jb}{R^2}\right)} \cdot \frac{1}{s^2}$$

$$K = \frac{mgd}{L\left(m + \frac{Jb}{R^2}\right)}$$

Now we plug in:

- $m = 0.064$
- $g = 9.81$
- $d = 0.0254$
- $L = 0.34$
- $Jb = 3.87 \times 10^{-7}$
- $R = 0.0127$

$$K \approx 0.72$$

## 2.4 Transfer Function

So, your transfer function becomes:

$$P(s) = \frac{X(s)}{\theta(s)} = \frac{0.72}{s^2}$$

### 3. System Analysis

Before designing a controller, we must first analyze how the system behaves on its own. In this section, we look at the 'Open-Loop' system, which means looking at how the ball moves if the motor does not automatically correct itself. This analysis proves mathematically that the system is unstable and confirms that a controller is necessary to keep the ball from falling off.

#### 3.1 Open-Loop Plant Model

Based on the mathematical model derived in Section 2, the open-loop transfer function of the ball and beam system is:

$$P(s) = \frac{X(s)}{\theta(s)} = \frac{0.72}{s^2}$$

This equation confirms that the system acts as a double integrator. By analyzing this transfer function, we can determine the system's stability and how it reacts to inputs before we add any control.

#### 3.2 Poles, Zeros, and Stability

From the open-loop transfer function, the pole-zero structure of the ball and beam plant is:

- **Poles:** a repeated pole at the origin,  $s = 0$  (multiplicity 2)
- **Zeros:** no finite zeros

The pole-zero map therefore consists of two coincident poles at the origin and no zeros. This means that the plant behaves as a pure double integrator. Internally, a double integrator is only marginally stable for zero input (the response to initial conditions does not blow up), but for any constant or step-like input in the beam angle the output grows without bound. In other words, the open-loop ball and beam plant is unstable in the BIBO sense and cannot by itself regulate the ball position.

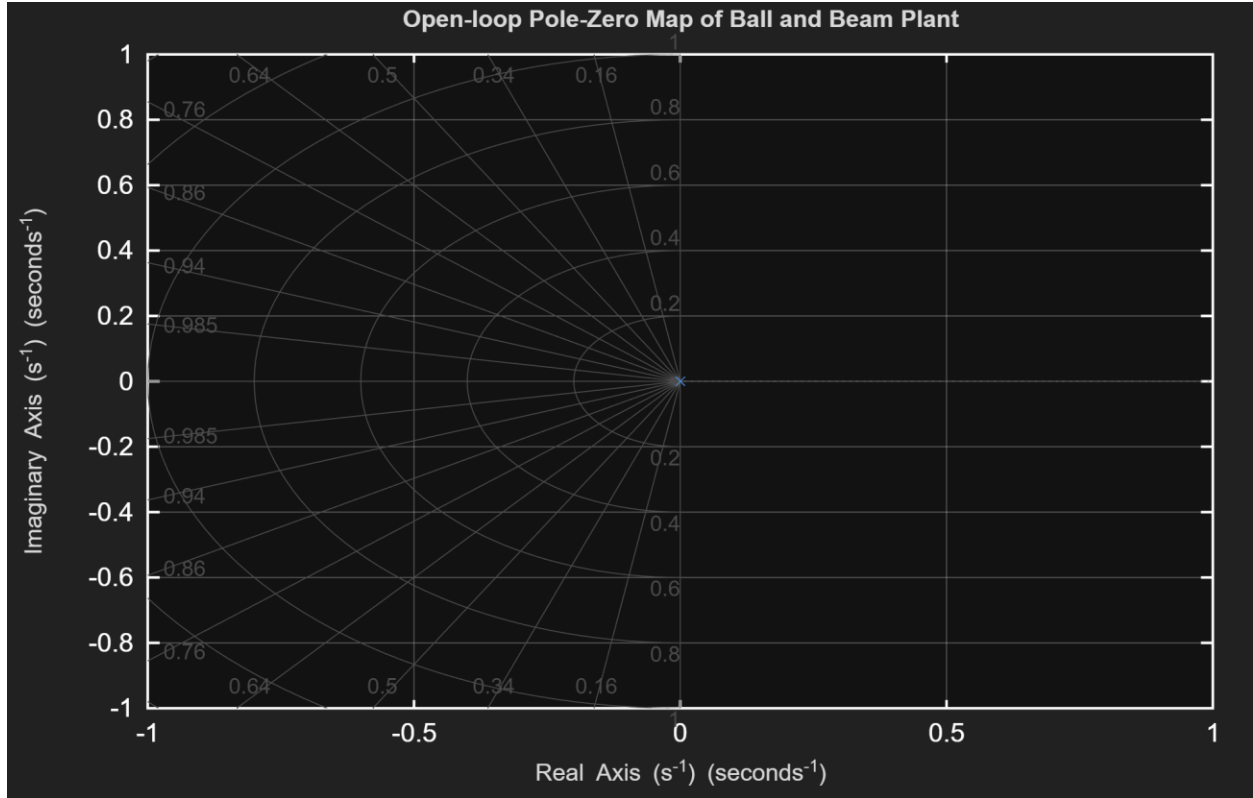


FIGURE 1: OPEN-LOOP POLE-ZERO MAP OF THE BALL AND BEAM PLANT SHOWING TWO COINCIDENT POLES AT THE ORIGIN.

As shown in Figure 1, both poles are located exactly at the origin and no zeros are present, which confirms the double-integrator nature of the plant. Any non-zero average torque applied through the beam angle will continuously accelerate the ball and drive it toward the end of the beam, so feedback control is essential to obtain a bounded and well-behaved position response.

### 3.3 Time-Domain Behavior to Angle Step Input

To better understand the time-domain behavior of the plant, consider a constant beam angle  $\theta_0$ . For the open-loop model  $P(s) = 0.72/s^2$ , the corresponding time-domain equation relating the ball position  $r(t)$  to the beam angle  $\theta(t)$  is

$$\ddot{r}(t) = K \theta_0,$$

where  $K = 0.72$  is the static gain obtained from the physical parameters of our setup. Integrating twice gives

$$r(t) = \frac{K \theta_0}{2} t^2 + C_1 t + C_2.$$

For zero initial position and velocity,  $C_1 = 0$  and  $C_2 = 0$ , so the ball position simplifies to

$$r(t) = \frac{K \theta_0}{2} t^2,$$

Which grows quadratically with time even for a small constant tilt of the beam. The corresponding open-loop step response obtained in MATLAB for a 1-radian step in beam angle is shown in Figure 2.

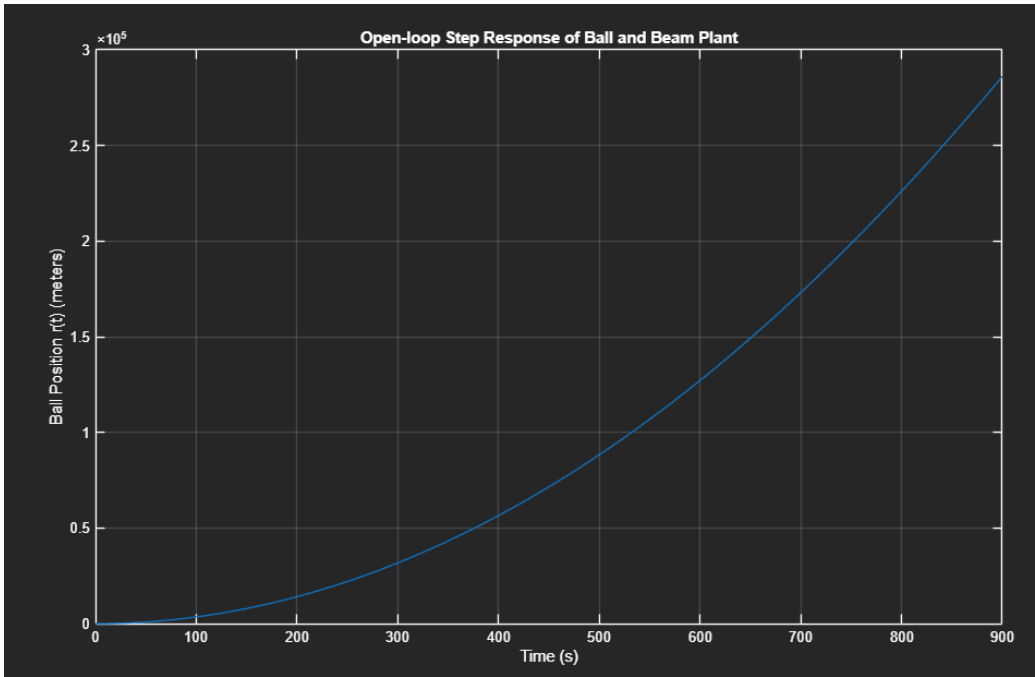


FIGURE 2: OPEN-LOOP STEP RESPONSE OF THE BALL AND BEAM PLANT FOR A 1-RADIAN STEP IN BEAM ANGLE USING THE SELECTED PARAMETER VALUES.

As seen in Figure 2, the ball position does not settle to a finite value; instead, its magnitude increases approximately proportionally to  $t^2$ . This unbounded response confirms that the open-loop ball and beam system is unstable for step inputs in the beam angle and cannot satisfy practical performance requirements without feedback control.

Consequently:

- There is **no finite settling time** for a step in  $\theta(t)$ .
- the **steady state value does not exist** for such step input.

This highlights that even a very small constant tilt of the beam will eventually drive the ball to the end of the beam, which is unacceptable for position regulation and strongly motivates the use of a stabilizing feedback controller.

### 3.4 System Type and Steady-State Error (Position Control Loop)

In terms of control theory, the plant  $P(s) = K/s^2$  contains **two integrators**. If we place this plant in a unity-feedback position-control configuration with a proper controller in the forward path, the resulting loop will be of **Type 2**. From the steady-state error analysis studied in the course, a Type-2 system has the following properties:

- **Step input in ball position:** zero steady-state error
- **Ramp input in ball position:** finite, non-zero steady-state error
- **Parabolic input:** infinite steady-state error

Therefore, once a stabilizing controller is added, the closed-loop ball position can, in principle, track step position commands with zero steady-state error, but the controller design must also ensure acceptable transient performance and robust stability.

## 4. PID Controller Design

### 4.1 Design Objective and Control Structure

The aim of this section is to design a feedback controller for the ball and beam system that satisfies the performance requirements from the Control Tutorials for MATLAB and Simulink (CTMS):

- Settling time  $T_s < 3s$ .
- Percent overshoot  $M_p < 5\%$  for a step change of 0.25m in ball position.

The open-loop plant relating the beam angle  $\Theta(s)$  to the ball position  $R(s)$  was derived in Section 2 as

$$P(s) = \frac{R(s)}{\Theta(s)} = -\frac{mgd}{L(\frac{J_b}{R^2} + m)} \frac{1}{s^2}$$

This plant is implemented in MATLAB and placed in a standard unity-feedback configuration, where the controller  $C(s)$  operates on the position error  $e(s) = R_{des}(s) - R(s)$ . The general PID controller considered has the form

$$C(s) = K_p + \frac{K_i}{s} + K_d s.$$

The task is to choose suitable gains  $K_p$ ,  $K_i$  and  $K_d$  such that the closed-loop step response meets the above specifications.

### 4.2 Proportional Control Evaluation

As a first attempt, a pure proportional controller  $C(s) = K_p$

was applied to the plant. The closed-loop transfer function was obtained in MATLAB using

$$G_{cl}(s) = \frac{K_p P(s)}{1 + K_p P(s)}$$

For several trial values of  $K_p$ , the corresponding step responses to a 0.25 m position command were simulated. The responses were oscillatory and lightly damped, with no choice of  $K_p$  yielding simultaneously a settling time below 3 s and an overshoot below 5%. Therefore, proportional control by itself is not sufficient to meet the required performance, and additional dynamics must be added to improve damping.

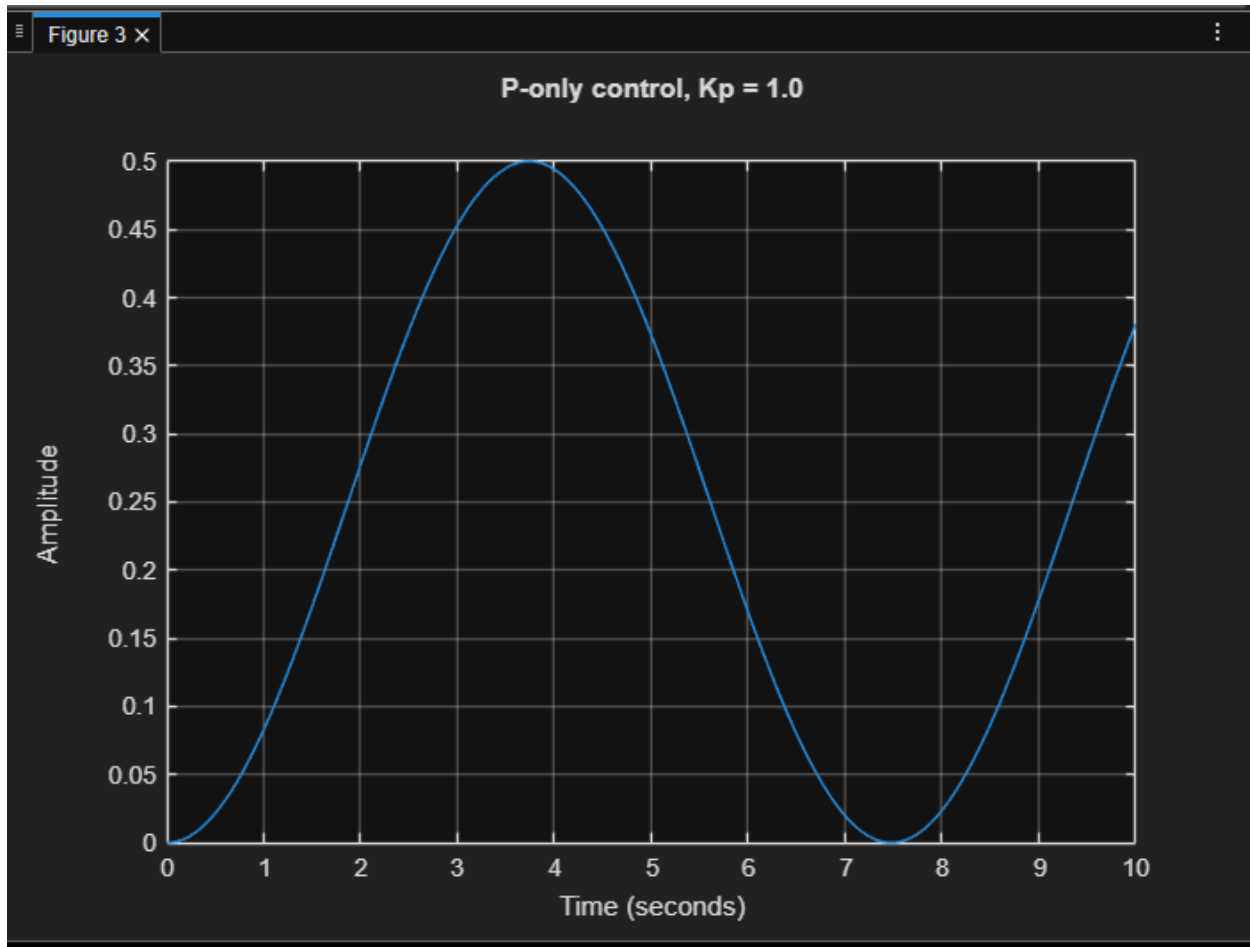


FIGURE 3:P-ONLY CONTROL

## 4.3 Proportional–Derivative Controller Design

### 4.3.1 Adding Derivative Action

To increase damping and stabilize the double-integrator dynamics, a derivative term was added, leading to a PD controller

$$C(s) = K_p + K_d s$$

The inclusion of the derivative term is expected to reduce overshoot and move the closed-loop poles further into the left half-plane, as discussed in the CTMS PID tutorial.

### 4.3.2 Tuning Procedure

The tuning procedure followed the same trial-and-error philosophy as in the CTMS example but using the physical parameters of our ball and beam setup. Initial gains were chosen as  $K_p=5$  and  $K_d=12$ , and the resulting closed-loop system was simulated for a 0.25 m step input. MATLAB's stepinfo function was used to extract the main time-domain performance indices.

The response for  $K_p=5$ ,  $K_d=12$  exhibited a fast rise, small overshoot and smooth approach to the final value. The performance metrics were

$$T_s = 2.416 \text{ s}, \quad M_p = 3.906$$

Both the settling time and overshoot satisfy the design criteria ( $T_s < 3$  and  $M_p < 5\%$ ). Because the response already meets the specifications and is well damped, further tuning of  $K_p$  and  $K_d$  was not required. The corresponding step-response plot is similar in shape to the final PD result shown in the CTMS tutorial but is obtained here using our own plant parameters and gain values.

Thus, the selected PD controller

$$K_p = 5, \quad K_d = 12$$

achieves the desired transient performance for the ball-position control problem.

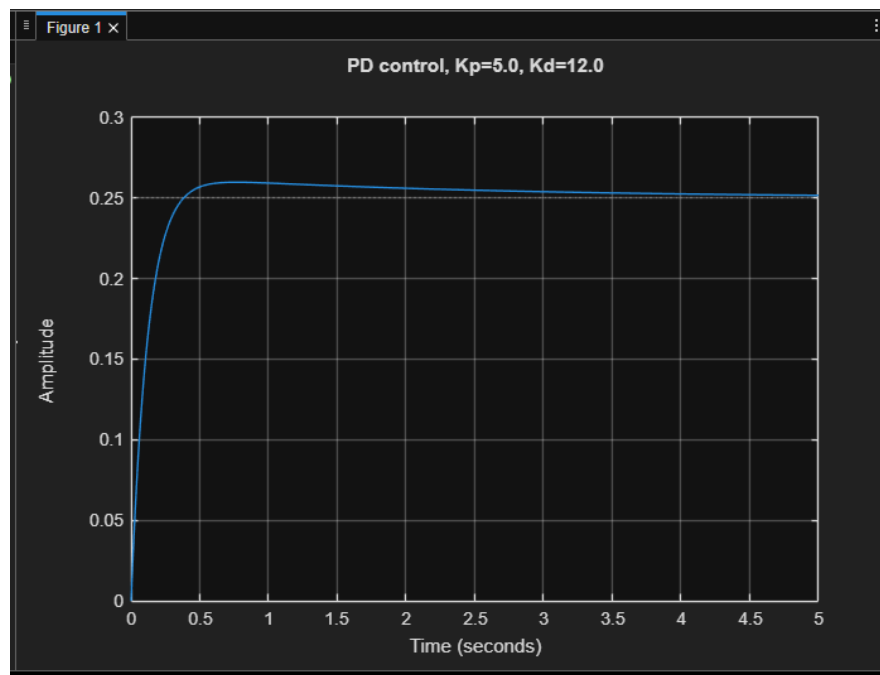


FIGURE 4: FINAL PD STEP RESPONSE

## 4.4 Justification for Zero Integral Gain

From the analysis in Section 3.4, the ball and beam plant contain two integrators, so when placed in a unity-feedback configuration with a proper stabilizing controller, the resulting system is Type 2. A Type-2 system has zero steady-state error for step inputs in the controlled variable.

Since the design objective is to track step changes in ball position, integral action is not necessary to remove steady-state error. Introducing a nonzero integral gain  $K_i$  would add another pole at the origin, potentially reducing phase margin and increasing overshoot without significant benefit for the specified task. For this reason, the integral term was set to zero, and a PD controller was adopted.

## 4.5 Final Controller

In summary, the final controller for the ball and beam system is a PD controller of the form

$$C(s) = K_p + K_d s = 5 + 12s,$$

implemented in a unity-feedback configuration with the plant  $P(s) \approx 0.72/s^2$ . The resulting closed-loop step response to a 0.25 m reference exhibits a settling time of approximately 2.42 s and an overshoot of about 3.9%, thereby satisfying all specified performance requirements without the need for integral action.

## 5. Root Locus Controller Design

### 5.1 Open-loop root locus

The open-loop transfer function of the ball and beam system can be written as

$$\frac{mgd}{L(J/R^2 + m)} = \frac{K}{s^2} = P(s)$$

where  $m$  is the ball mass,  $R$  is the ball radius,  $L$  is the beam length,  $d$  is the gear distance,  $J$  is the beam moment of inertia, and  $g$  is the gravitational acceleration.

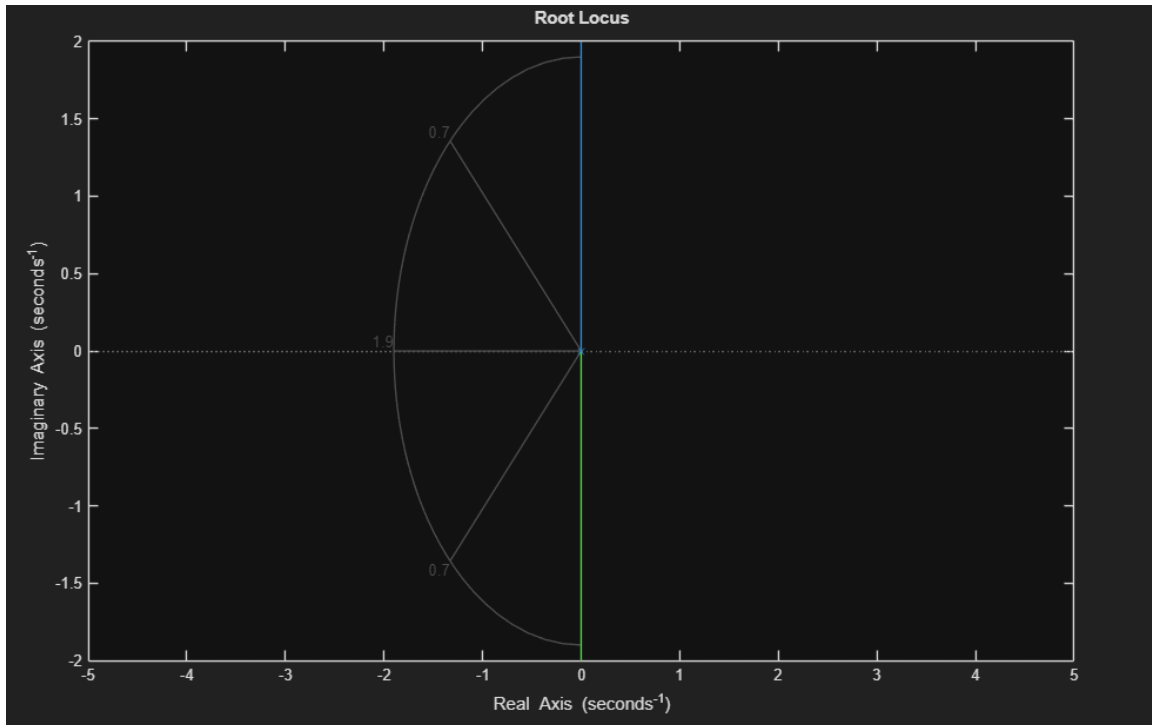


FIGURE 5: OPEN-LOOP ROOT LOCUS WITH DESIGN SPECS

The corresponding open-loop root locus of  $P(s)$  is shown in Fig. 5. The locus starts from a double pole at the origin and lies on the imaginary axis for all positive values of the gain, which confirms that the uncompensated plant is unstable.

The design specifications for this project are a settling time  $T_s < 3$  and a percent overshoot  $M_p < 5\%$ .

These requirements are represented on the root locus using the constant-damping lines  $\zeta=0.7$  and the constant-frequency circle  $\omega_n = 1.9$  rad/s, as illustrated in Fig. 5. Since no part of the open-loop locus enters the shaded design region defined by these limits, a simple proportional controller cannot satisfy the transient-response specifications.

## 5.2 Lead-compensator design

To shift the closed-loop poles toward the desired region in the left-half plane, a lead compensator of the form

$$\frac{0s + z}{0s + p} = C(s)$$

was introduced, with  $z_0 = 0.01$  and  $p_0 = 5$ . The zero is placed close to the origin, while the pole is located farther to the left on the real axis. This configuration bends the root locus of the compensated plant  $C(s)P(s)$  toward the left-half plane and allows portions of the locus to pass through the design region.

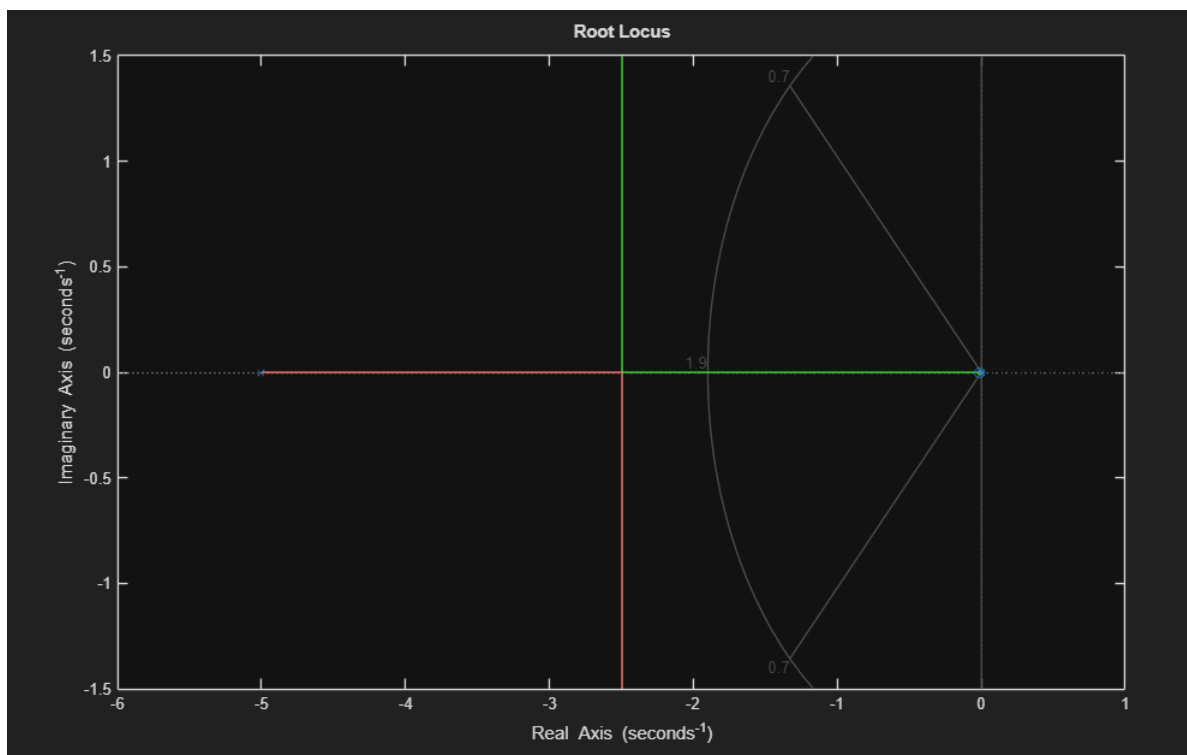


FIGURE 6: ROOT LOCUS OF COMPENSATED PLANT WITH LEAD CONTROLLER.

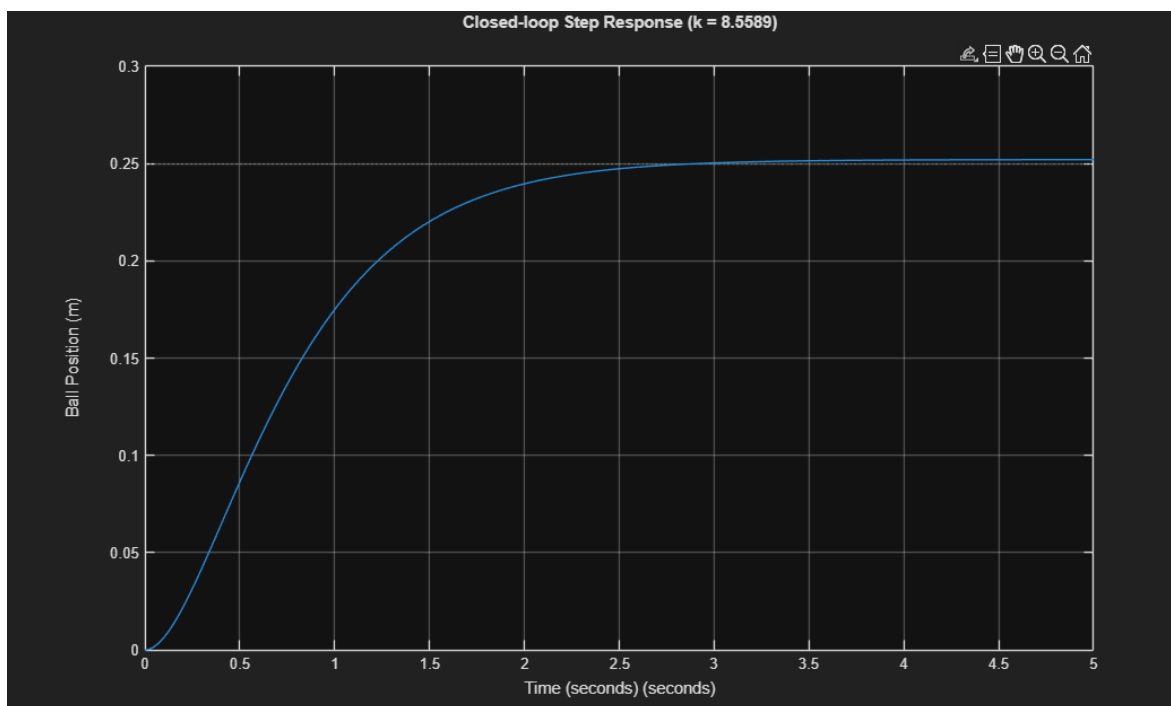
The root locus of the compensated system is plotted in **Fig. 6.** in this figure, several points on the locus now lie inside the region associated with  $\zeta=0.7$  and  $T_s < 3$ , indicating that an appropriate choice of gain can meet the transient-performance requirements.

The feedback gain for the compensated system was selected using the MATLAB command [ `rlocfind` ] applied to the root locus of  $C(s)P(s)$ .

A point on the locus inside the design region, approximately at  $s_{sel} = -2.01$  was chosen interactively. This point corresponds to a gain  $k = 8.55$  which yields the following closed-loop poles:  $s_1 = -2.98$ ,  $s_2 = -2.01$ ,  $s_3 = -0.01$ . All poles are in the left-half plane, so the compensated system is stable.

The unity-feedback closed-loop transfer function is

$$\frac{k C(s) P(s)}{k C(s) P(s) + 1} = T(s)$$



**FIGURE 7: CLOSED-LOOP STEP RESPONSE**

The step response of the closed-loop system for a 0.25-m reference command is shown in Fig. 7. From the MATLAB [ `stepinfo` ] results, the response has a settling time of approximately  $T_s = 2.4s$  and essentially zero percent overshoot  $M_p = 0\%$

Both values satisfy the design specifications  $T_s < 3$  and  $M_p < 5\%$ . Therefore, the lead-compensated controller designed via the root-locus method successfully achieves the desired transient performance for the ball-and-beam system.

## 6. Simulink

In this section, all the above modeling, analysis and controller design for the ball and beam model will be implemented in Simulink and simulated.

### 6.1 ball-and-beam model

Figure 8 shows the model of the Ball and Beam system. It takes the beam angle as an input and calculates the forces acting on the ball, such as gravity. The large rectangular box contains the mathematical equation that turns these forces into acceleration. The two integrators then convert that acceleration into speed and finally into the ball's position. The arrows looping back allow the system to constantly update itself based on where the ball is at that exact moment.

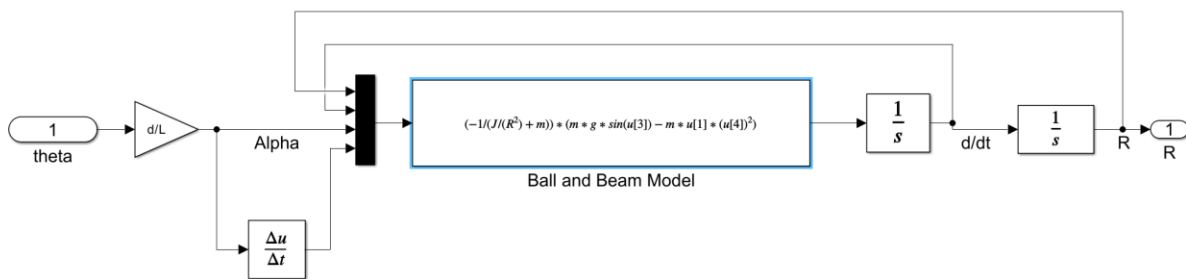


FIGURE 8: BALL AND BEAM MODEL IN SIMULINK

### 6.2 ball-and-beam controller

In Figure 9, the ball and beam will be considered as a subsystem, then comparing three different subsystems' outputs open looped, closed looped with and without a controller. The controller used was derived from the root locus section above that gives  $k$  value for the system 8.5 and lead compensation usage with zero at -0.01 and pole at -5. All systems will have a unit step input of 0.25 and outputs will be simultaneously simulated to compare the differences between the subsystems.

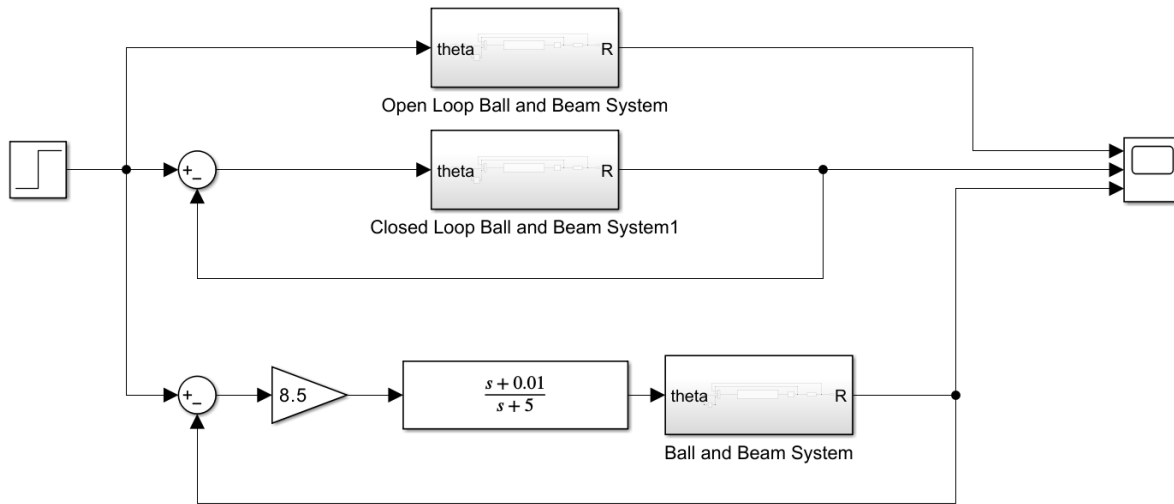


FIGURE 9: OPEN, CLOSE AND CONTROLLED SYSTEM, IN SIMULINK

## 6.3 ball and beam simulated results

In Figure 10 shows three outputs charts colored green, blue and red, the green chart is the open loop system, which is going to infinity, therefore considered unstable, then the blue output is the uncompensated closed loop system which shows sinusoidal output meaning a marginally stable performance, going from the left side of the beam to the right. Lastly, the red, which is the controlled system, reaching under 3 seconds a steady value of 0.25, which is the required output from the system to ensure that the ball will be stable on the beam and in the required position.

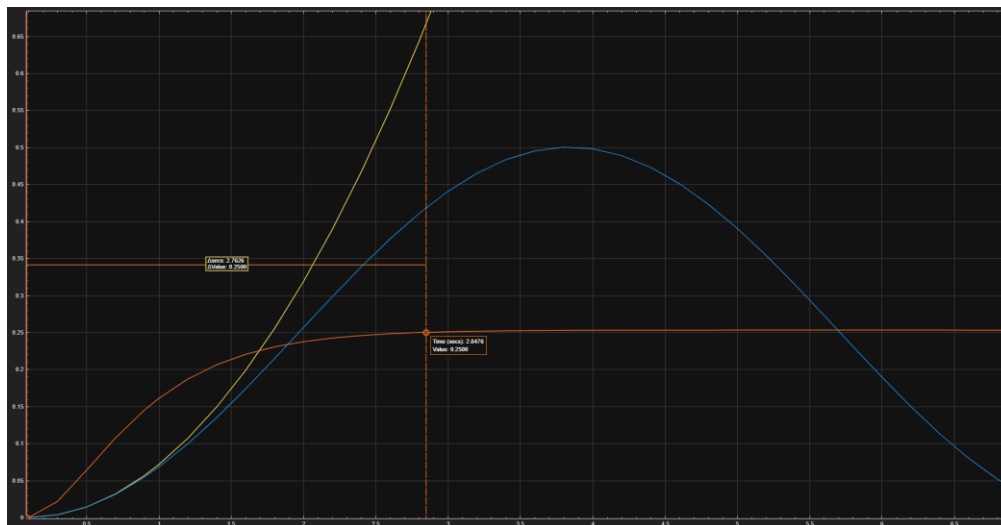


FIGURE 10: OPEN, CLOSE, AND CONTROLLED SYSTEM RESPONSE GRAPHS

## 7. Conclusion

In this project, the ball-and-beam system was selected as a representative real-life unstable dynamic system that satisfies the second-order requirement of the EE331 course project. A linearized mathematical model was derived using realistic physical parameters from an actual experimental setup, and the resulting transfer function was used as the basis for analysis. The detailed open-loop analysis, including pole–zero examination, time-domain behavior, stability, and steady-state error characteristics, showed that the plant behaves as a pure double integrator with a repeated pole at the origin. This leads to marginal stability and an unbounded position response for constant beam angles, confirming that the uncontrolled system cannot meet practical performance specifications.

To address these limitations, two controller-design approaches were investigated. First, a PID-based design was carried out, and it was shown that proportional control alone cannot satisfy the settling-time and overshoot requirements. By introducing derivative action and tuning the gains, a PD controller was obtained that achieves a settling time below 3 s and an overshoot below 5% for a 0.25 m step in ball position. Second, a lead-compensator controller was designed using the root-locus method. The compensator shifts the closed-loop poles into a region of the complex plane with adequate damping and speed, also meeting the same performance specifications. MATLAB and Simulink simulations were used throughout to verify the analytical predictions and to compare uncompensated and compensated responses.

Overall, the project successfully integrated modelling, detailed analysis, PID/PD design, root-locus lead-compensator design, and simulation into a coherent study of a practical control problem. The results clearly illustrate the importance of feedback and compensator design when dealing with inherently unstable or marginally stable systems. They also provide a solid foundation for future work, such as robustness analysis, implementation on real hardware, or comparison with alternative control strategies for the ball-and-beam system.

## 8. References

- [1] QUANSER, QUANSER, [Online]. Available:  
[https://www.se.rit.edu/~se463/ResearchProject/Quanser/SRV02\\_Exp3\\_Ball%20&%20Beam.pdf](https://www.se.rit.edu/~se463/ResearchProject/Quanser/SRV02_Exp3_Ball%20&%20Beam.pdf).  
[Accessed 1 12 2025].
- [2] M. H. R. Shirke. [Online]. Available: <https://www.ijert.org/research/mathematical-modeling-simulation-and-control-of-ball-and-beam-system-IJERTV4IS030879.pdf>. [Accessed 1 12 2025].
- [3] A. T. Ali. [Online]. Available: <https://pubs.sciepub.com/acis/3/1/1/index.html>. [Accessed 2 12 2025].

## Appendix A

| <i>Member No.</i> | <i>Name</i>                 | <i>Summary of work</i>                                                                                                                                                                                                                                                                                                                                                                                  | <i>Percentage of work</i> |
|-------------------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| #1                | Ahmed Al-Harthy             | built the Simulink subsystems for the ball and beam plant and simulated the outputs for the open-loop, closed-loop, and compensated systems on Simulink. Compared three graphs to analyze performance, wrote the Simulink section of the report, and handled the final formatting for submission.                                                                                                       | 20%                       |
| #2                | Naif Al-Suhaim              | Designed and implemented the PID controller for the ball-and-beam system. This included modeling the plant in MATLAB, evaluating proportional control, tuning a PD controller to meet the required settling time and overshoot specifications, and providing justification for the final controller parameters.                                                                                         | 20%                       |
| #3                | Abdullah Mustafa Alaidaroos | Performed the Detailed Analysis for the ball-and-beam system, derived the approximate open-loop transfer function, generated the MATLAB pole-zero and step-response plots, and wrote the project conclusion summarizing the main results.                                                                                                                                                               | 20%                       |
| #4                | Masoud Saad Albogami        | Designed and implemented the root-locus-based lead compensator for the ball-and-beam system. This included constructing the open-loop root locus, applying the design specifications in the s-plane, selecting an appropriate lead compensator and feedback gain in MATLAB, validating the closed-loop settling time and overshoot, and writing the Root Locus Controller Design section of the report. | 20%                       |
| #5                | Abdullah Fallatah           | Derived the mathematical model of the ball-and-beam system. This included describing the physical mechanism, establishing the kinematic relationship between the motor and beam angles, formulating the equation of motion, calculating the system constants using physical parameters, and deriving the final open-loop transfer function presented in the System Modeling section                     | 20%                       |

# Appendix B

## System analysis code

```
1  s = tf('s');
2  K = -0.706;
3  P_ball = K / s^2;
4
5  figure;
6  step(P_ball);
7  title('Open-loop Step Response of Ball and Beam Plant');
8  xlabel('Time (s)');
9  ylabel('Ball Position r(t)');
10 grid on;
11 |
```

```
1  s = tf('s');
2  K = -0.706;
3  P_ball = K / s^2;
4
5  figure;
6  pzmap(P_ball);
7  title('Open-loop Pole-Zero Map of Ball and Beam Plant');
8  xlabel('Real Axis (s^{-1})');
9  ylabel('Imaginary Axis (s^{-1})');
10 grid on;
11
```

## PID code

```
/MATLAB Drive/untitled.m
1  % --- Physical Parameters ---
2  m = 0.064;
3  R = 0.0127;
4  g = -9.81;
5  L = 0.34;
6  d = 0.0254;
7  J = 3.87e-7;
8
9  s = tf('s');
10
11 % --- Plant model ---
12 P_ball = -m*g*d/L/(J/R^2 + m) / s^2;
13
14
15 % 1) Proportional-only control
16
17 Kp = 1; % Proportional gain
18 C_P = pid(Kp); % P controller (Ki = 0, Kd = 0)
19
20 sys_P = feedback(C_P * P_ball, 1);
21
22 figure;
23 t = 0:0.01:10;
24 step(0.25 * sys_P, t);
25 grid on;
26 title(sprintf('P-only control, Kp = %.1f', Kp));
27 xlabel('Time (s)');
28 ylabel('Ball position (m)');
29
30
31 % 2) Final PD controller
32
33 Kp = 5;
34 Kd = 12;
35 Ki = 0;
36 C_PD = pid(Kp, Ki, Kd);
37
38 sys_PD = feedback(C_PD * P_ball, 1);
39
40 figure;
41 t = 0:0.01:5;
42 step(0.25 * sys_PD, t);
43 grid on;
44 title(sprintf('PD control, Kp = %.1f, Kd = %.1f', Kp, Kd));
45 xlabel('Time (s)');
46 ylabel('Ball position (m)');
47
48 % Step-response performance
49 info = stepinfo(0.25 * sys_PD);
50 info.SettlingTime
51 info.Overshoot
```

## Root Locus Controller Design Code

```
1      L = 0.34;
2      d = 0.0254;
3      m = 0.064;
4      R = 0.0127;
5      J = 3.87e-7;
6      g = -9.81;
7
8      s = tf('s');
9      P_ball = -m*g*d / L / (J/R^2 + m) / s^2;
10
11     figure
12     rlocus(P_ball)
13     sgrid(0.70, 1.9)
14     axis([-5 5 -2 2])
15     title('Open-loop Root Locus of P_ball')
16     grid on
17
18     zo = 0.01;
19     po = 5;
20     C = tf([1 zo],[1 po]);
21
22     figure
23     rlocus(C*P_ball)
24     sgrid(0.70, 1.9)
25     axis([-5 1 -1.5 1.5])
26     title('Root Locus with Lead Compensator C(s)')
27     grid on
28
29     [k, poles] = rlocfind(C*P_ball);
30
31     sys_cl = feedback(k*C*P_ball, 1);
32
33     t = 0:0.01:5;
34     figure
35     step(0.25*sys_cl, t)
36     grid on
37     title(sprintf('Closed-loop Step Response (k = %.4f)', k))
38     xlabel('Time (seconds)')
39     ylabel('Ball Position (m)')
40
41
42     info = stepinfo(0.25*sys_cl, t);
43     Ts = info.SettlingTime
44     Mp = info.Overshoot
45
```